

AI Extension with Playwright Typescript Implementation

Implementation through agent mode in copilot

The screenshot displays the Copilot Chat interface within a code editor. The chat window is titled "CHAT" and shows a conversation about implementing a Playwright TypeScript extension. The main heading in the chat is "CODEBASE ANALYSIS FOR SELENIUM JAVA AND PLAYWRIGHT TYPESCRIPT". A button labeled "Proceed with implementation step by step" is visible. The chat content includes the following text:

Updated 3 files and planned implementation steps

Perfect! I'll implement step by step. Let me start by applying all the changes efficiently.

Created 6 todos

STEP 1: Adding Playwright TypeScript Prompts to `prompts.js`

Let me add the two new prompt templates:

Read `prompts.js`, lines 180 to 220

- Preparing

Todos (1/6)

- ☒ Add Playwright TypeScript prompts to `prompts.js`
- ☐ Update `CODE_GENERATOR_TYPES` export
- ☐ Update `getPromptKeys()` method in `chat.js`
- ☐ Add `isTypescriptPlaywright()` helper method
- ☐ Update error message method
- ☐ Validate all changes work together

Below the todos, there is a section for a new task with a button "+", a label "Untitled-1", and a text input "Describe what to build". At the bottom of the chat window, there are controls for the agent: "+", "Agent", "Claude Haiku 4.5", and a "Run" button. The status bar at the bottom of the editor shows "Ln 1, Col 1", "Spaces: 4", "UTF-8", "CRLF", "{ } Plain Text", and icons for file explorer, search, and chat.

CHAT

+

▼

⚙️

...

|

🗨️

✕

←

CODEBASE ANALYSIS FOR SELENIUM JAVA AND PLAYWRIGHT TYPESCRIPT

📄

Updated chat.js +15 -5

✅

STEP 2 Complete: Updated `getPromptKeys()` method, added `isTypescriptPlaywright()` helper, and updated error message.

Validated all changes worked together

✅

STEP 3: Validation - Verify all changes

Let me verify the changes were applied correctly:

📄

Read `prompts.js`, lines 175 to 195

📄

Read `prompts.js`, lines 235 to 260

•

Let me fix the startLine parameter - it should be a number without the newline escape.

📄

Read `chat.js`, lines 555 to 630

•

Loading

▼

Todos (6/6)

⌵

✅

Add Playwright TypeScript prompts to prompts.js

✅

Update CODE_GENERATOR_TYPES export

✅

Update getPromptKeys() method in chat.js

✅

Add isTypescriptPlaywright() helper method

✅

Update error message method

🔵

Validate all changes work together

>

2 files changed +158 -5

Keep

Undo

📄

+

📄

Untitled-1

Assignment description:

Implement playwright/typescript with vibe coding

URL Used: <https://test.salesforce.com/>

Settings Screenshot:



Code Generator

Settings

Language Binding

TypeScript



Browser Engine

Playwright



Select LLM Provider

Choose provider



Select Model

Choose model



Testleaf

Extensions

Login | Salesforce

salesforce logo png download

Salesforce Logo PNG Transpare

test.salesforce.com



Salesforce login

Username

Password

Log In to Sandbox

☐ Remember me

[Forgot Your Password?](#) [Use Custom Domain](#)

© 2026 Salesforce, Inc. All rights reserved. | [Privacy](#)

San Francisco | April 15-16, 2026

Stream TDX live, the can't-miss developer conference of the year.

Agentic AI is changing the game and Agentforce is leading the way. Join us for three days of deep technical sessions and virtual hands-on trainings. Experience the future of software and learn how to build it.

[Register for free](#)



AI Code Generator

Code Generator

Settings

☒ Feature File ☒ Page Class

Reset Chat

```
const loginButton = page.locator('input')
await expect(loginButton).toBeVisible()
await loginButton.click();
});

/**
 * Verify successful login.
 * The sandbox redirects to a URL contain
 * Adjust selectors as needed for the act
 */
then('I should be logged in successfully'
// wait for navigation after login
await page.waitForLoadState('networkidle')

// Example verification: URL contains "/"
await expect(page).toHaveURL(/.*\/home.

// Example verification: user avatar is
const avatar = page.locator('img.avatar')
await expect(avatar).toBeVisible();
});
```

Provide additional information

Testleaf

Extensions

Login | Salesforce

salesforce logo png download

Salesforce Logo PNG Transpare

test.salesforce.com



Salesforce login

Username

Password

Log In to Sandbox

☐ Remember me

[Forgot Your Password?](#) [Use Custom Domain](#)

© 2026 Salesforce, Inc. All rights reserved. | [Privacy](#)

San Francisco | April 15-16, 2026

Stream TDX live, the can't-miss developer conference of the year.

Agentic AI is changing the game and Agentforce is leading the way. Join us for three days of deep technical sessions and virtual hands-on trainings. Experience the future of software and learn how to build it.

[Register for free](#)



AI Code Generator

Code Generator

Settings

Language Binding

TypeScript

Browser Engine

Playwright

Select LLM Provider

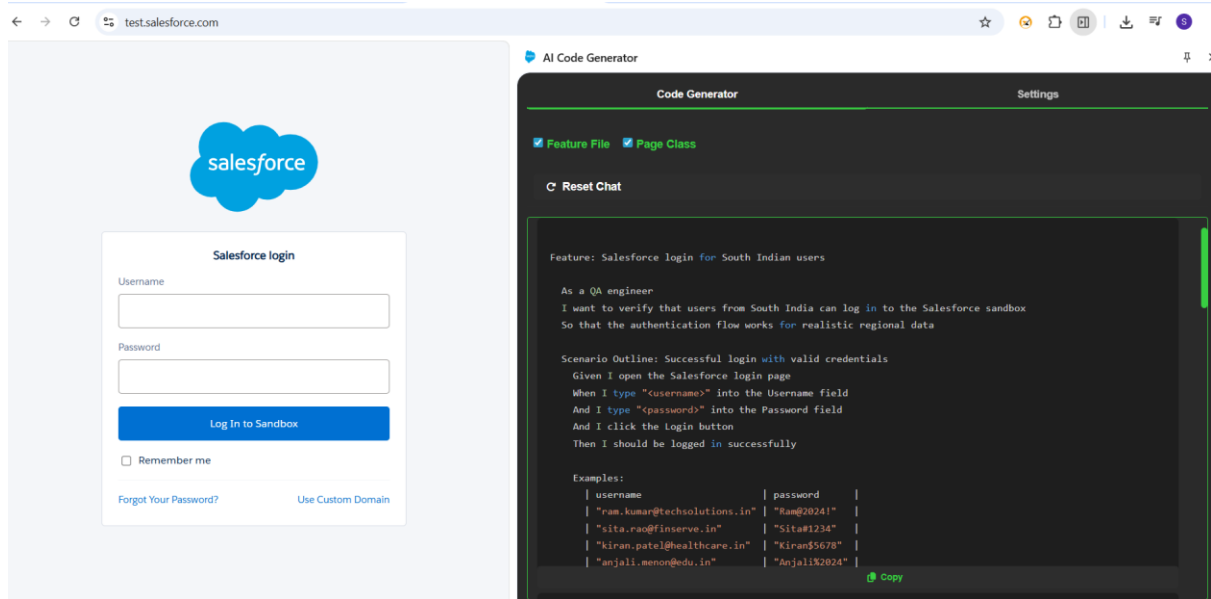
Groq

Select Model

openai/gpt-oss-120b

Groq API Key

Output Screenshot :-



```
import { Before, After, Given, When, Then } from '@cucumber/cucumber';
import { chromium, Browser, BrowserContext, Page, expect } from '@playwright/test';

// Global Playwright objects
let browser: Browser;
let context: BrowserContext;
let page: Page;

/**
 * Launch a new Chromium browser before each scenario.
 * Headless mode can be toggled via the `HEADLESS` environment variable.
 */
Before(async function () {
  const headless = process.env.HEADLESS !== 'false';
  browser = await chromium.launch({ headless });
  context = await browser.newContext();
  page = await context.newPage();
});
```

```

/**
 * Close the browser after each scenario to free resources.
 */
After(async function () {
    await context.close();
    await browser.close();
});

/**
 * Navigate to the Salesforce login URL.
 */
Given('I open the Salesforce login page', async function () {
    await page.goto('https://test.salesforce.com/');
    // Ensure the login form is visible before proceeding
    await expect(page.locator('form#login_form')).toBeVisible();
});

/**
 * Fill the Username input identified by its unique id.
 */
When('I type {string} into the Username field', async function (username: string) {
    const usernameInput = page.locator('input#username');

```

```

    // Fill the Username field
    await usernameInput.fill(username);
});

/**
 * Fill the Password input identified by its unique id.
 */
When('I type {string} into the Password field', async function (password: string) {
    const passwordInput = page.locator('input#password');
    await expect(passwordInput).toBeVisible();
    await passwordInput.fill(password);
});

/**
 * Click the Login button. The button is an <input> element with id="Login".
 */
When('I click the Login button', async function () {
    const loginButton = page.locator('input#Login');
    await expect(loginButton).toBeVisible();
    await loginButton.click();
});

```

```
When('I click the Login button', async function () {
  const loginButton = page.locator('input#Login');
  await expect(loginButton).toBeVisible();
  await loginButton.click();
});

/**
 * Verify successful login.
 * The sandbox redirects to a URL containing "/home" and displays a user-specific avatar.
 * Adjust selectors as needed for the actual post-login page.
 */
Then('I should be logged in successfully', async function () {
  // Wait for navigation after login
  await page.waitForLoadState('networkidle');

  // Example verification: URL contains "/home"
  await expect(page).toHaveURL(/.*\/home.*/);

  // Example verification: user avatar is visible
  const avatar = page.locator('img.avatar, img[alt*="User"]');
  await expect(avatar).toBeVisible();
});
```

Provide additional information

 Inspect

Generate